

SUPPLY MANAGEMENT SYSTEM

Task Register

Product Variants, Dynamic Attributes & Downstream Impact

Maps to FSD Addendum 26 | Includes test data for every task | ASP.NET Core 8 | EF Core 8

Version 1.0 | 2026

FSD source	FSD Addendum 26 - Product Variants, Dynamic Attributes & Downstream Impact
Total tasks	9
Total estimated dev days	26 days (~6 working weeks)
Critical priority tasks	5
Total test cases (with embedded test data)	106
Epics	6 epics: schema/migration, attributes engine, FK migration, search, UI, end-to-end
Test data approach	Every task includes TEST DATA entries with exact values (SKUs, prices, quantities, attribute values) so QA can reproduce assertions precisely
Key worked example	Dell Latitude 5450 with 3 variants + OPC Cement as a simple product - traced through PO, GRN, Inventory, MIR, MIV, Ledger, Search across every task

Table of Contents

Table of Contents 2

1. FSD Traceability Summary 3

2. Master Test Data Reference 4

 2.1 Products and Variants 4

 2.2 Dell Latitude Variants..... 4

 2.3 Laptop Category Attributes 4

 2.4 Dell i7 Variant Attribute Values 4

 2.5 Search Index Expected Content 5

3. Detailed Task Register 6

4. Sequencing Notes..... 18

1. FSD Traceability Summary

Every task references FSD Addendum 26 sections. Test data values are consistent across all tasks - the same Dell Latitude variants and Cement product are used throughout the entire register so test assertions chain correctly.

Task ID	Title	Days	FSD Ref	Priority
PV-001	Migration: drop transactional data, revise Product table, create ProductVariants entity	3d	Sec 2.1, 3.1, 9.1	Critical
PV-002	AttributeDefinitions, CategoryAttributes, and VariantAttributeValues schema and CRUD API	3d	Sec 4.1, 4.2, 4.3	Critical
PV-003	Dynamic UI form renderer and Admin category-attribute configuration screen	3d	Sec 5, 6	High
PV-004	Purchase Order and GRN: change product_id to variant_id with two-level picker	3d	Sec 7.1, 7.2	Critical
PV-005	Inventory and MIR: change product_id to variant_id across InventoryItems, Stock Ledger, MIR, MIV, Reservations	3d	Sec 7.3, 7.4	Critical
PV-006	ProductSearchIndex, Full-Text Search, and barcode lookup	3d	Sec 8, 11	High
PV-007	Product management screen with integrated variant CRUD and the reusable product-variant picker	3d	Sec 2, 3, 5	High
PV-008	POS variant integration and report updates	2d	Sec 7.5, 7.6	High
PV-009	End-to-end integration test: full procurement cycle at variant level	3d	All sections	Critical
TOTAL	9 tasks	26d		

2. Master Test Data Reference

The following test data is used consistently across all tasks. QA should create this data exactly as specified in the test environment. Values are referenced by task ID where they are first created.

2.1 Products and Variants

Product	product_code	Category	Variants	Created in
Dell Latitude 5450	DELL-LAT-5450	Laptop	3 variants (see below)	PV-001
OPC Cement 50kg	CEMENT-OPC	Building Material	1 default variant (auto)	PV-001
HP EliteBook 850	HP-EB-850	Laptop	2 variants (1 default + 1 added)	PV-007

2.2 Dell Latitude Variants

SKU	Variant Name	Purchase	Selling	Barcode	Default?
DELL-5450-I5-8-256	i5 / 8GB / 256GB	85,000	95,000	8901234567890	No
DELL-5450-I5-16-512	i5 / 16GB / 512GB	105,000	120,000	8901234567891	No
DELL-5450-I7-16-512	i7 / 16GB / 512GB	135,000	155,000	8901234567892	Yes

2.3 Laptop Category Attributes

attribute_name	display_name	data_type	control	options (if DROPDOWN)	required	searchable
cpu	CPU	DROPDOWN	DROPDOWN	Intel i3, i5, i7, i9, Ryzen 5, Ryzen 7	Yes	Yes
ram	RAM	DROPDOWN	DROPDOWN	4GB, 8GB, 16GB, 32GB, 64GB	Yes	Yes
storage	Storage	DROPDOWN	DROPDOWN	128/256/512GB SSD, 1/2TB SSD	Yes	Yes
screen_size	Screen Size	DECIMAL	NUMBERBOX	n/a	No	No
color	Color	DROPDOWN	DROPDOWN	Silver, Black, White, Gray, Blue	No	Yes
gpu	GPU	TEXT	TEXTBOX	n/a	No	Yes

2.4 Dell i7 Variant Attribute Values

Attribute	Value	Validated against
cpu	Intel i7	DROPDOWN options list
ram	16GB	DROPDOWN options list
storage	512GB SSD	DROPDOWN options list
screen_size	15.6	DECIMAL type validation
color	Silver	DROPDOWN options list
gpu	Intel Iris Xe	TEXT (free text, no validation)

2.5 Search Index Expected Content

For Dell i7 variant, the searchable_text field should contain: 'Dell Latitude 5450 i7 / 16GB / 512GB SSD DELL-5450-I7-16-512 8901234567892 Intel i7 16GB 512GB SSD Silver Intel Iris Xe'

Searches that should MATCH this variant: 'Dell i7', 'i7 16GB', '512GB SSD', '8901234567892', 'Iris Xe', 'Latitude Silver'

Searches that should NOT match: 'Ryzen', 'HP', '256GB', 'Black'

3. Detailed Task Register

Epic 1 - Schema, Migration, and Product Variant Entities (FSD Sections 2, 3, 9)

Drop existing product data, create revised Product table, ProductVariants entity, and the auto-default-variant mechanism. This epic is the foundation that every other task depends on.

PV-001	Critical	3d	Sec 2.1, 3.1, 9.1	Migration: drop transactional data, revise Product table, create ProductVariants entity
--------	----------	----	-------------------	---

Execute the destructive migration (project in development - no production data at risk). Remove price/barcode columns from Product, create ProductVariants with all 16 fields, and implement the auto-default-variant creation rule.

Development tasks	Test cases (with test data)
- Create migration script respecting FK order: TRUNCATE POLines, PurchaseOrders, GRNLines, GRNs, Invoices, MIRLines, MIR, MIVLines, MIV, StockReservation, InventoryItems, InventoryLedgerEntries, MasterProductLedger, MasterFinancialLedger, SupplierLedgerEntries, SupplierPayments, ProjectCostLedger, DeptCostLedger, Wastage, MaterialReturn, MaterialConsumption, Products - ALTER Product table: DROP columns purchase_price, selling_price, barcode, last_purchase_price. KEEP product_id, organization_id, product_code, product_name, category_id, sub_category_id, description, brand, unit, is_active, is_batch_tracked, is_serial_tracked, is_direct_consumption, reorder_point, created_at, created_by - CREATE ProductVariants entity with 16 fields per FSD Section 3.1: variant_id (PK), product_id (FK), organization_id, sku (UQ per org), variant_name, barcode (UQ nullable per org), purchase_price, selling_price, last_purchase_price, weight, dimensions, is_default, is_active, reorder_point, sort_order, created_at/by - EF Core migration: dotnet ef migrations add ProductVariantSchema - ProductService.CreateAsync: when a product is created, auto-create one default variant (is_default=true) with variant_name = product_name, sku = product_code + '-DEFAULT', purchase_price from the request body - Validation: exactly one variant per product must have is_default = true. Enforce via FluentValidation on create/update - Indexes: IX_ProductVariants_SKU (UQ filtered by org_id), IX_ProductVariants_Barcode (UQ nullable filtered by org_id), IX_ProductVariants_ProductId	- TEST DATA - Product: product_code='DELL-LAT-5450', product_name='Dell Latitude 5450', category='Laptop', brand='Dell', unit='Piece' - TEST DATA - Variant 1: sku='DELL-5450-I5-8-256', variant_name='i5 / 8GB / 256GB', purchase_price=85000, selling_price=95000, barcode='8901234567890', is_default=false - TEST DATA - Variant 2: sku='DELL-5450-I7-16-512', variant_name='i7 / 16GB / 512GB', purchase_price=135000, selling_price=155000, barcode='8901234567892', is_default=true - TEST DATA - Simple product: product_code='CEMENT-OPC', product_name='OPC Cement 50kg', category='Building Material'. On creation auto-generates variant: sku='CEMENT-OPC-DEFAULT', variant_name='OPC Cement 50kg', purchase_price=1200, is_default=true - POST /api/products with Dell Latitude data creates Product row + 2 Variant rows with correct FKs - POST /api/products with Cement data (no explicit variants) auto-creates one default variant - Attempting to set is_default=false on ALL variants of a product returns 400 (at least one must be default) - SKU uniqueness: creating variant with sku='DELL-5450-I5-8-256' in same org twice returns 409 Conflict - Barcode uniqueness: two variants in same org with barcode='8901234567890' returns 409 - After migration: Products table has zero rows, all transactional tables have zero rows, Product table no longer has purchase_price/selling_price/barcode columns

Acceptance criteria	- Migration drops all data cleanly with no orphaned FK references - ProductVariants auto-created for simple products - Uniqueness constraints enforced on SKU and barcode per tenant
---------------------	--

Epic 2 - Dynamic Attributes Engine (FSD Sections 4, 6)

AttributeDefinitions, CategoryAttributes, VariantAttributeValues entities, the Admin category-attribute configuration screen, and the variant attribute CRUD API.

PV-002

Critical

3d

Sec 4.1, 4.2, 4.3

AttributeDefinitions, CategoryAttributes, and VariantAttributeValues schema and CRUD API

Create the three attribute entities, seed sample attributes for Laptop and Building Material categories, and build the full API for attribute management, category-attribute linking, and variant value storage.

Development tasks	Test cases (with test data)
- CREATE AttributeDefinitions entity (14 fields per FSD Section 4.1): attribute_id, organization_id, attribute_name (UQ per org), display_name, data_type (TEXT/NUMBER/DECIMAL/DATE/BOOLEAN/DROPDOWN/MULTI_SELECT), control_type (TEXTBOX/NUMBERBOX/DATEPICKER/TOGGLE/DROPDOWN/MULTI_SELECT/TEXTAREA), dropdown_options (JSON), default_value, validation_regex, is_required, is_searchable, is_filterable, sort_order, is_active - CREATE CategoryAttributes entity (6 fields per FSD Section 4.2): category_attribute_id, category_id (FK), attribute_id (FK), is_required (override), display_order, organization_id - CREATE VariantAttributeValues entity (5 fields per FSD Section 4.3): value_id, variant_id (FK), attribute_id (FK), value (NVARCHAR(500)), organization_id - EF Core migration: dotnet ef migrations add DynamicAttributes - CRUD APIs: GET/POST/PUT /api/attributes, GET/POST/DELETE /api/categories/{id}/attributes, GET/PUT for variant attribute values within the variant endpoints - Validation: when data_type=DROPDOWN, validate that submitted value exists in dropdown_options JSON array. When data_type=NUMBER, validate value parses to int. When validation_regex is set, validate value matches. - Seed sample attributes for SCM-DEMO org (see test data below)	- TEST DATA - Seed 6 attributes for Laptop category: - attr: cpu, display: CPU, type: DROPDOWN, control: DROPDOWN, options: ['Intel i3','Intel i5','Intel i7','Intel i9','AMD Ryzen 5','AMD Ryzen 7'], required: true, searchable: true - attr: ram, display: RAM, type: DROPDOWN, control: DROPDOWN, options: ['4GB','8GB','16GB','32GB','64GB'], required: true, searchable: true - attr: storage, display: Storage, type: DROPDOWN, control: DROPDOWN, options: ['128GB SSD','256GB SSD','512GB SSD','1TB SSD','2TB SSD'], required: true, searchable: true - attr: screen_size, display: Screen Size, type: DECIMAL, control: NUMBERBOX, required: false, searchable: false - attr: color, display: Color, type: DROPDOWN, control: DROPDOWN, options: ['Silver','Black','White','Gray','Blue'], required: false, searchable: true - attr: gpu, display: GPU, type: TEXT, control: TEXTBOX, required: false, searchable: true - TEST DATA - Seed 3 attributes for Mobile category: - attr: storage (reuse), ram (reuse), imei_required: type BOOLEAN, control: TOGGLE, required: true - TEST DATA - Seed VariantAttributeValues for Dell i7 variant: cpu='Intel i7',

	ram='16GB', storage='512GB SSD', color='Silver', screen_size='15.6' • POST /api/attributes creates attribute with correct data_type and dropdown_options • POST /api/categories/{laptopId}/attributes links cpu attribute to Laptop category with display_order=1, is_required=true • Submitting variant value 'Intel i99' for cpu (DROPDOWN) returns 400 - value not in options • Submitting variant value 'abc' for screen_size (DECIMAL) returns 400 - not a valid decimal • GET /api/categories/{laptopId}/attributes returns 6 attributes in correct display_order • Attribute in Org A is invisible to Org B (tenant isolation verified)
--	---

Acceptance criteria	• All 7 data types validated correctly at API level • Category-attribute linking with per-category required override works • Dropdown validation enforces options list strictly • Tenant isolation complete for attributes
---------------------	--

PV-003	High	3d	Sec 5, 6	Dynamic UI form renderer and Admin category-attribute configuration screen
--------	------	----	----------	--

Build the React dynamic form component that renders attribute controls based on category metadata, and the Admin screen for configuring which attributes appear on which category with drag-and-drop ordering.

Development tasks	Test cases (with test data)
• React: DynamicAttributeForm component accepts categoryId prop, calls GET /api/categories/{categoryId}/attributes on mount, renders controls dynamically per attribute's control_type • Control mapping: TEXT/TEXTAREA -> text input/textarea, NUMBER/DECIMAL -> number input, DATE -> date picker, BOOLEAN -> toggle switch, DROPDOWN -> select dropdown populated from dropdown_options, MULTI_SELECT -> multi-select chips • Required attributes show red asterisk. Client-side validation mirrors server-side: type checking, regex, required enforcement	• TEST SCENARIO: Select category 'Laptop' on product form -> form renders 6 fields: CPU (dropdown with i3/i5/i7/i9/Ryzen5/Ryzen7), RAM (dropdown with 4/8/16/32/64), Storage (dropdown), Screen Size (number input), Color (dropdown), GPU (text input). CPU, RAM, Storage show red asterisks. • TEST SCENARIO: Select category 'Mobile' -> form renders: Storage, RAM, IMEI Required (toggle). CPU and Screen Size do NOT appear (not linked to Mobile). • TEST SCENARIO: Select category 'Building Material' -> form renders zero attribute fields (no attributes linked to this category).

- Integrate DynamicAttributeForm into the variant create/edit form: appears below the standard variant fields (SKU, price, barcode). On edit, pre-populate values from VariantAttributeValues - On category change on the parent Product form: clear existing attribute form and reload for the new category - React: Admin screen at Inventory > Categories > {name} > Configure Attributes (/inventory/categories/{id}/attributes) - Admin screen: left panel shows available attributes (all AttributeDefinitions not yet linked to this category), right panel shows linked attributes with drag-and-drop reordering for display_order. Per-attribute toggle for is_required override. Live preview panel showing the rendered form. - Save button calls PUT /api/categories/{id}/attributes with the full ordered list	- TEST SCENARIO: Change category from Laptop to Mobile -> Laptop attributes clear, Mobile attributes load - Admin drags GPU from position 6 to position 2 -> display_order updates to 2, other attributes shift down - Admin toggles color from optional to required for Garment category -> red asterisk appears in preview - Live preview correctly shows the rendered form matching what end users will see - Submitting the variant form with required cpu field empty returns validation error
--	--

Acceptance criteria	- Dynamic form renders correctly for every control type without category-specific code - Admin can configure attributes per category with immediate effect - Category change correctly reloads the attribute form
---------------------	---

Epic 3 - FK Migration: product_id to variant_id Across All Modules (FSD Section 7)

The single most impactful change: every transactional line table that referenced product_id now references variant_id. Covers PO, GRN, Inventory, MIR, MIV, Stock Ledger, and Master Product Ledger.

PV-004	Critical	3d	Sec 7.1, 7.2	Purchase Order and GRN: change product_id to variant_id with two-level picker
--------	----------	----	--------------	---

Change POLines and GRNLines FK from product_id to variant_id. Build the two-level product-then-variant picker for PO line entry. GRN lines auto-inherit variant_id from the PO line.

Development tasks	Test cases (with test data)
- ALTER POLines: rename product_id to variant_id, change FK target from Products to ProductVariants - ALTER GRNLines: rename product_id to variant_id, change FK target from Products to ProductVariants - EF Core migration: dotnet ef migrations add POGRNVariantFK - Update PurchaseOrderService: POLine creation accepts variant_id. Unit_price defaults from ProductVariant.purchase_price. - Update GrnService: GRN line creation from a PO auto-populates variant_id from the PO line. Manual GRN line entry also uses variant_id. - React PO line form: two-level picker. Step 1: searchable product dropdown (shows product_name + product_code). Step 2: variant dropdown (shows variant_name + sku + purchase_price). For products with only one variant (is_default=true, single variant), auto-select and hide the variant dropdown. - GRN barcode scan: GET /api/variants/lookup?barcode={code} resolves to variant_id. If scanned variant does not match the PO line's variant_id, show warning but allow override. - On GRN approval: update ProductVariant.last_purchase_price = GRN line's accepted unit price	- TEST DATA - PO-2026-00001 for supplier 'Lenovo Distributors': - Line 1: variant_id = DELL-5450-I5-8-256, qty=10, unit_price=85000 - Line 2: variant_id = DELL-5450-I7-16-512, qty=5, unit_price=135000 - PO grand_total = 10*85000 + 5*135000 = 1,525,000 - TEST DATA - GRN-2026-00001 against PO-2026-00001: - Line 1: variant_id inherited from PO = DELL-5450-I5-8-256, qty_received=10, qty_accepted=10 - Line 2: variant_id inherited from PO = DELL-5450-I7-16-512, qty_received=5, qty_accepted=5 - TEST DATA - PO for Cement (simple product): - Line 1: variant_id = CEMENT-OPC-DEFAULT, qty=100, unit_price=1200. Variant picker hidden - user sees only 'OPC Cement 50kg' - POST PO with variant_id for Dell i5 variant: unit_price auto-populates to 85,000 - Creating GRN against PO: lines auto-populate with correct variant_ids from PO lines - Barcode scan '8901234567892' resolves to Dell i7 variant and matches PO line 2 - Barcode scan '999999999999' (non-existent) returns 404 - Barcode scan of variant NOT on the PO shows warning but allows override - GRN approval updates ProductVariant(DELL-5450-I7-16-512).last_purchase_price to the GRN's accepted unit price - Simple product (Cement): PO line form shows only the product name, no variant picker visible

Acceptance criteria	- PO lines correctly reference variant_id with accurate price population - GRN lines inherit variant_id from PO automatically - Barcode scan resolves to the correct specific variant - Simple products show no variant picker to the user
---------------------	--

PV-005	Critical	3d	Sec 7.3, 7.4	Inventory and MIR: change product_id to variant_id across InventoryItems, Stock Ledger, MIR, MIV, Reservations
--------	----------	----	--------------	--

Change every inventory and material-issue related FK from product_id to variant_id. Stock is now tracked per variant per warehouse. MIR/MIV lines reference variant_id.

Development tasks	Test cases (with test data)
- ALTER InventoryItems: rename product_id to variant_id, change FK. Add unique constraint: IX_InventoryItems_VariantId_WarehouseId (one row per variant per warehouse) - ALTER InventoryLedgerEntries: rename product_id to variant_id, change FK - ALTER MasterProductLedger: rename product_id to variant_id, change FK - ALTER MaterialIssueRequestDetail (MIR lines): rename product_id to variant_id - ALTER MaterialIssueDetail (MIV lines): rename product_id to variant_id - ALTER StockReservation: rename product_id to variant_id - EF Core migration: dotnet ef migrations add InventoryMIRVariantFK - Update IInventoryLedgerService.CreateEntryAsync: accept variant_id instead of product_id - Update IStockAvailabilityService.GetAvailability(variantId, warehouseId): query InventoryItem by variant_id + warehouse_id - Update IMasterProductLedgerService.PostMovementAsync: accept variant context, denormalise product_name from variant's parent product - React MIR line form: two-level picker identical to PO (product then variant, simple products auto-select) - Product-level stock summary: add GET /api/products/{id}/stock-summary that computes SUM(qty_on_hand), SUM(qty_reserved), SUM(qty_available) across all variants of the product (computed, not stored)	- TEST DATA - After GRN-2026-00001 is approved: - InventoryItem row 1: variant_id=DELL-5450-I5-8-256, warehouse_id=MAIN-WH, qty_on_hand=10, qty_reserved=0, qty_available=10 - InventoryItem row 2: variant_id=DELL-5450-I7-16-512, warehouse_id=MAIN-WH, qty_on_hand=5, qty_reserved=0, qty_available=5 - InventoryLedgerEntry: variant_id=DELL-5450-I5-8-256, type=GRN_RECEIPT, quantity_in=10 - TEST DATA - MIR-2026-00001 for Project 'Tower Block A': - Line 1: variant_id=DELL-5450-I7-16-512, requested_qty=3 - On approval: StockReservation created with variant_id=DELL-5450-I7-16-512, reserved_qty=3 - InventoryItem: qty_reserved becomes 3, qty_available becomes 2 - TEST DATA - MIV-2026-00001 against MIR-2026-00001: - Line 1: variant_id=DELL-5450-I7-16-512, issued_qty=3 - After issue: qty_on_hand=2, qty_reserved=0, qty_available=2 - InventoryLedgerEntry: variant_id=DELL-5450-I7-16-512, type=MATERIAL_ISSUE, quantity_out=3 - GET /api/products/{delId}/stock-summary returns: total_on_hand=12, total_reserved=0, total_available=12 (sum across both variants before MIR) - Stock availability check for variant DELL-5450-I7-16-512 at MAIN-WH returns correct qty_available=5 - MIR approval for 6 units of DELL-5450-I7-16-512 (only 5 available) returns 400 with live availability figures - MasterProductLedger entry for GRN receipt shows source_type=SUPPLIER, product_name='Dell Latitude 5450 (i7/16GB/512GB)'

Acceptance criteria	- Inventory tracked at variant+warehouse level with correct qty_on_hand/reserved/available - Stock availability, reservation, and issue all operate on variant_id - Product-level summary correctly aggregates across variants - All ledger entries reference variant_id with denormalised product info
---------------------	--

Epic 4 - Product Search Index (FSD Section 8)

Denormalised ProductSearchIndex with SQL Server Full-Text Search, Hangfire rebuild, and barcode exact-match lookup.

PV-006	High	3d	Sec 8, 11	ProductSearchIndex, Full-Text Search, and barcode lookup
--------	------	----	-----------	--

Build the denormalised ProductSearchIndex table, the Hangfire job that rebuilds it on variant changes, the full-text search API endpoint, and the barcode exact-match lookup for POS/GRN.

Development tasks	Test cases (with test data)
- CREATE ProductSearchIndex table: search_entry_id, variant_id (FK), product_id (FK), organization_id, product_name, product_code, sku, barcode, variant_name, category_name, brand, searchable_text (NVARCHAR(MAX) - concatenation of product_name + variant_name + all searchable attribute values) - EF Core migration: dotnet ef migrations add AddProductSearchIndex - Create SQL Server Full-Text catalog and index on ProductSearchIndex.searchable_text - Implement IProductSearchIndexService.RebuildForVariantAsync(variantId): loads variant, parent product, all VariantAttributeValues where attribute.is_searchable=true, concatenates into searchable_text, upserts into ProductSearchIndex - Implement IProductSearchIndexService.RebuildAllAsync(): rebuilds for all active variants (used for initial population and Admin-triggered full rebuild) - Wire RebuildForVariantAsync to fire via Hangfire BackgroundJob.Enqueue on variant create/update and on VariantAttributeValue create/update - GET /api/products/search?q={query} - uses CONTAINS or FREETEXT against searchable_text, returns matched variants with parent product info, highlighted matching terms, and attribute values. Paginated, tenant-scoped. - GET /api/variants/lookup?barcode={code} - exact match on ProductVariants.barcode (indexed), returns variant with product info. Used by POS barcode scanner and GRN receiving.	- TEST DATA - ProductSearchIndex row for Dell i7 variant: - searchable_text = 'Dell Latitude 5450 i7 / 16GB / 512GB SSD DELL-5450-I7-16-512 8901234567892 Intel i7 16GB 512GB SSD Silver' (concatenation of: product_name + variant_name + sku + barcode + cpu value + ram value + storage value + color value) - TEST DATA - Search query 'Dell i7' should match the above row (both 'Dell' and 'i7' appear in searchable_text) - TEST DATA - Search query 'Ryzen' should NOT match any Dell variant (no Ryzen in attributes) - TEST DATA - Search query '8901234567892' should match via barcode in searchable_text - TEST DATA - Barcode lookup '8901234567892' returns variant DELL-5450-I7-16-512 directly - Search 'Dell i7' returns Dell Latitude i7/16GB/512GB variant with highlighted 'Dell' and 'i7' - Search 'Dell' returns ALL Dell variants (i5-8-256, i5-16-512, i7-16-512) - Search 'Cement' returns OPC Cement 50kg default variant - Search with empty query returns paginated list of all active variants - Barcode lookup for non-existent barcode returns 404 - Updating a variant's color attribute from Silver to Black triggers Hangfire rebuild; subsequent search for 'Black' returns the updated variant - Search results from Org A do not include Org B's products (tenant isolation)

Acceptance criteria

- Full-text search correctly finds variants by product name, variant name, SKU, barcode, and searchable attribute values
- Barcode lookup is instant (indexed exact match)

	• Search index automatically maintained via Hangfire on variant/attribute changes • Tenant isolation enforced on search results
--	---

Epic 5 - Product and Variant Management Screens (FSD Sections 2, 3, 5)

Product CRUD screen with integrated variant management, the two-level product-variant picker component reused across PO/GRN/MIR/POS, and the product detail screen with variant list.

PV-007	High	3d	Sec 2, 3, 5	Product management screen with integrated variant CRUD and the reusable product-variant picker
--------	------	----	-------------	--

Build the Product create/edit screen that includes variant management inline, the variant detail form with dynamic attributes, and the reusable two-level picker component used by PO/GRN/MIR/POS.

Development tasks	Test cases (with test data)
- React: Product list screen (/inventory/products) showing product families with variant_count column, category, brand, is_active status - React: Product create form: product_code, product_name, category (dropdown), sub_category, brand, unit, description, batch/serial/direct-consumption toggles, reorder_point. On save, auto-creates one default variant. - React: Product detail screen: product info at top, variant list below as a table showing sku, variant_name, purchase_price, selling_price, barcode, is_default badge, is_active status. 'Add Variant' button. - React: Variant create/edit form (modal or slide-over): sku, variant_name, purchase_price, selling_price, barcode, weight, dimensions, is_default, reorder_point, sort_order. Below these fields: the DynamicAttributeForm component (PV-003) rendering category-specific attributes. - React: Reusable ProductVariantPicker component: Step 1 dropdown searches products by name/code, Step 2 dropdown shows variants of selected product with sku + variant_name + price. For single-variant products (is_default only), auto-select and hide Step 2. This component is used by PO line form, GRN line form, MIR line form, and POS. - Variant deletion: if variant has been transacted (any PO/GRN/MIR/Inventory reference), soft-delete (is_active=false). If never transacted, allow hard delete. - Default variant toggle: changing is_default on one variant auto-removes it from the previous default (only one default per product)	- TEST SCENARIO: Create product 'HP EliteBook 850': product_code='HP-EB-850', category='Laptop'. System auto-creates default variant sku='HP-EB-850-DEFAULT'. Product detail shows 1 variant. - TEST SCENARIO: Add variant to HP EliteBook: sku='HP-EB-850-I7-32-1T', variant_name='i7 / 32GB / 1TB', purchase_price=195000, selling_price=220000. Dynamic form shows Laptop attributes (CPU, RAM, Storage, etc.). Fill in: CPU='Intel i7', RAM='32GB', Storage='1TB SSD'. Save. Product detail now shows 2 variants. - TEST SCENARIO: On PO line form, ProductVariantPicker: type 'HP' -> shows 'HP EliteBook 850' in Step 1 dropdown. Select it -> Step 2 shows 2 variants. Select 'i7/32GB/1TB' -> variant_id set, unit_price=195000. - TEST SCENARIO: On PO line form, type 'Cement' -> shows 'OPC Cement 50kg'. Select it -> Step 2 auto-selects default variant and hides. unit_price=1200. - Deleting a variant that exists on a PO line soft-deletes (is_active=false), variant remains visible on existing PO - Deleting a never-transacted variant hard-deletes the row - Changing is_default from variant A to variant B: only variant B is default, A loses the flag

Acceptance criteria

- Product create auto-generates default variant
- Variant management inline on product detail screen
- ProductVariantPicker reusable across PO/GRN/MIR/POS with correct auto-select for simple products
- Soft-delete protects transacted variants

Epic 6 - POS Integration, Reports Update, and End-to-End Verification (FSD Sections 7.5, 7.6, 8, 12)

POS variant support (if POS module exists), update all reports to group by variant, and full end-to-end test of the complete procurement-to-issue cycle at variant level.

PV-008	High	2d	Sec 7.5, 7.6	POS variant integration and report updates
--------	------	----	--------------	--

If POS module is implemented: update SalesLine to reference variant_id, barcode scan resolves to variant. Update all existing reports to show variant_name/sku alongside product_name. Update supplier scorecard and cost allocation to use variant_id.

Development tasks	Test cases (with test data)
- If POS exists: ALTER SalesLines product_id to variant_id. Barcode scan calls GET /api/variants/lookup?barcode={code} and adds variant to sale. Price from ProductVariant.selling_price. - If POS exists: stock decrement on sale uses variant_id + warehouse_id (POS store's warehouse) - Update all report endpoints to include variant_name, sku alongside product_name in response DTOs: Stock Level Summary, Stock Movement Report, Material Issue Register, Material Consumption Register, Project Consumption, Department Consumption, Wastage Report - Supplier Scorecard (Addendum 23): GRN scoring already operates at PO/GRN level - verify it works correctly with variant_id on GRN lines (quantity accuracy checks compare variant-level qty_received vs PO variant-level qty_ordered) - Cost allocation (Addendum 20): MIV line now carries variant_id. unit_cost comes from InventoryItem for that variant. Cost posting amount calculation unchanged: qty * unit_cost. Update narration to include variant_name. - Master Product Ledger entries: denormalise variant_name and sku into the entry alongside product_name for readability	- TEST DATA - POS sale (if POS exists): scan barcode '8901234567892' -> resolves to DELL-5450-I7-16-512, adds to sale at selling_price=155,000 - TEST DATA - Stock Level Summary report for Dell Latitude: shows 2 rows (one per variant), each with variant_name, sku, qty_on_hand, unit_cost, total_value - TEST DATA - Material Issue Register: MIV entry shows 'Dell Latitude 5450 - i7/16GB/512GB' not just 'Dell Latitude 5450' - TEST DATA - Project Consumption: cost entry for Tower Block A shows variant-level detail: 'i7/16GB/512GB x 3 @ 135,000 = 405,000' - Stock Level report correctly shows separate rows per variant with accurate per-variant quantities - Stock Movement report shows variant_name and sku in each movement row - Supplier Scorecard quantity accuracy: PO ordered 5 of Dell i7 variant, GRN received 5 -> 100% match -> 25 points

Acceptance criteria	- POS barcode scan resolves to variant with correct selling_price - All reports show variant-level detail alongside product name - Cost allocation correctly posts per variant - No existing report is broken by the FK change
---------------------	---

PV-009	Critical	3d	All sections	End-to-end integration test: full procurement cycle at variant level
--------	----------	----	--------------	--

A single comprehensive integration test that exercises the ENTIRE procurement-to-issue-to-ledger cycle using variant-level data, verifying every module works correctly together after the FK migration.

Development tasks	Test cases (with test data)
- Build an automated integration test (xUnit) that executes the full cycle in sequence, asserting at each step: - Step 1 - SETUP: Create category 'Laptop' with 6 attributes (cpu, ram, storage, screen_size, color, gpu). Create product 'Dell Latitude 5450' with 3 variants (i5/8/256, i5/16/512, i7/16/512) and full attribute values. Verify ProductSearchIndex contains all 3 variants. - Step 2 - SEARCH: Search for 'Dell i7' -> returns exactly 1 result (the i7 variant). Search for 'Dell' -> returns 3 results. - Step 3 - PO: Create PO for Lenovo Distributors with 2 lines: 10x i5/8/256 @ 85,000 and 5x i7/16/512 @ 135,000. Verify grand_total = 1,525,000. - Step 4 - GRN: Create GRN against the PO. Lines auto-inherit variant_ids from PO. Approve GRN. Verify: InventoryItem for i5/8/256 at Main WH has qty_on_hand=10. InventoryItem for i7/16/512 has qty_on_hand=5. InventoryLedgerEntries created per variant. ProductVariant.last_purchase_price updated. MasterProductLedger entries with source=Supplier, destination=Warehouse. - Step 5 - MIR: Create MIR for Project 'Tower Block A', line: 3x i7/16/512. Submit and approve through the workflow. Verify: StockReservation created for variant i7/16/512, reserved_qty=3. InventoryItem.qty_reserved=3, qty_available=2. - Step 6 - MIV: Create Material Issue Voucher against the MIR. Issue 3x i7/16/512. Verify: qty_on_hand=2, qty_reserved=0, qty_available=2. InventoryLedgerEntry type=MATERIAL_ISSUE, variant_id=i7/16/512, quantity_out=3. ProjectCostLedger entry: 3 x 135,000 = 405,000. - Step 7 - VERIFY AGGREGATES: GET /api/products/{deIID}/stock-summary -> total_on_hand=12 (10 of i5 + 2 remaining of i7). Stock Level report shows 2 variant rows. Master Product Ledger shows 3 entries (2 GRN receipts + 1 material issue). Timeline (Addendum 21) shows the full chain. - Step 8 - SIMPLE PRODUCT: Repeat steps 3-6 for 'OPC Cement 50kg' (simple product, default variant, no variant picker shown) to verify the auto-default-variant path works identically.	- ASSERTION: After Step 2 - search 'Dell i7' returns variant_id=DELL-5450-I7-16-512 with searchable_text containing 'Intel i7 16GB 512GB SSD' - ASSERTION: After Step 3 - PO.grand_total = 1,525,000 with 2 lines referencing correct variant_ids - ASSERTION: After Step 4 - InventoryItem(DELL-5450-I5-8-256, MAIN-WH).qty_on_hand = 10 - ASSERTION: After Step 4 - InventoryItem(DELL-5450-I7-16-512, MAIN-WH).qty_on_hand = 5 - ASSERTION: After Step 4 - ProductVariant(DELL-5450-I7-16-512).last_purchase_price = 135,000 - ASSERTION: After Step 5 - StockReservation.variant_id = DELL-5450-I7-16-512, reserved_qty = 3 - ASSERTION: After Step 5 - InventoryItem(i7).qty_available = 2 (5 on_hand - 3 reserved) - ASSERTION: After Step 6 - InventoryItem(i7).qty_on_hand = 2 (5 minus 3 issued) - ASSERTION: After Step 6 - ProjectCostLedger entry amount = 405,000 - ASSERTION: After Step 7 - product stock summary total_on_hand = 12 - ASSERTION: After Step 8 - Cement PO/GRN/MIR cycle completes identically using the auto-default-variant with no variant picker interaction - ASSERTION: Entire test completes within 30 seconds (performance baseline)

Acceptance criteria

- Complete procurement-to-issue cycle works at variant level with correct stock, ledger, and cost figures
- Simple product (default variant) path works identically without variant picker
- All aggregates (stock summary, ledger totals, cost entries) are consistent across all modules
- Performance baseline: full cycle under 30 seconds

4. Sequencing Notes

This addendum has a strict sequential dependency chain because each epic builds on the schema and data from the previous one.

Dependency	Reason
PV-001 before everything	All tasks depend on ProductVariants entity and the revised Product table existing.
PV-002 before PV-003	Dynamic form renderer needs AttributeDefinitions and CategoryAttributes to exist.
PV-001 before PV-004 and PV-005	FK migration needs ProductVariants table to change FKs to.
PV-004 before PV-005	Inventory FK migration depends on PO/GRN FK migration being done (test data flows PO -> GRN -> Inventory).
PV-002 before PV-006	Search index concatenates searchable attribute values which must exist.
PV-001 + PV-002 + PV-003 can parallel PV-004	Attribute engine is independent of FK migration if coordinated on shared schema.
PV-005 before PV-008	Report updates read from variant-level InventoryItems which must be migrated first.
All tasks before PV-009	End-to-end test exercises every module and needs everything in place.

PV-002 and PV-003 (attribute engine) can be developed in parallel with PV-004 (PO/GRN FK migration) by two different developers, merging before PV-006 (search) which needs both.

- End of Document -